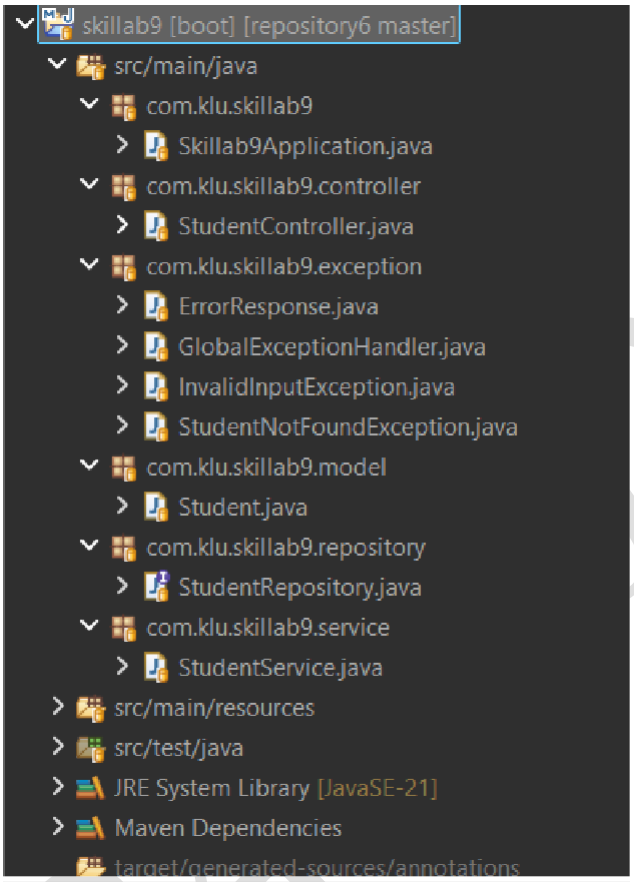
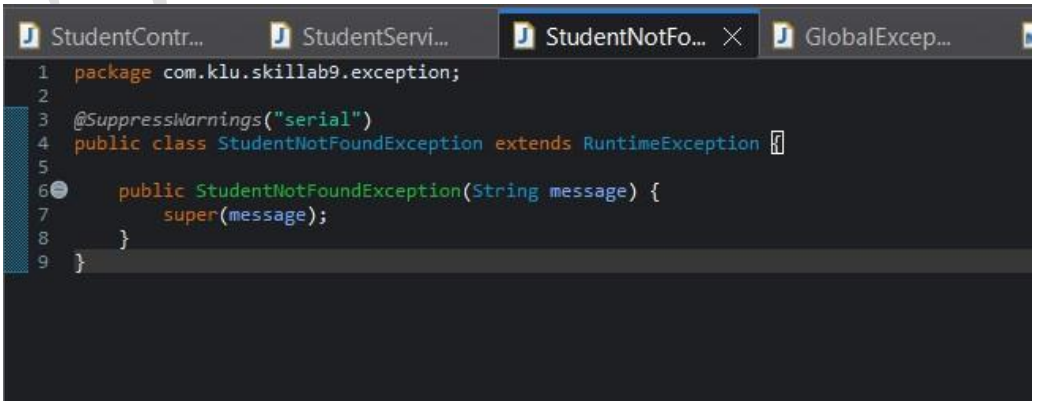
NAME:	R.Hemalatha
ID:	2400030211
SECTION:	S-59
SUBJECT:	FULL STACK APPLICATION DEVELOPMENT

#SKILL - 9 Global Exception Handling using @ControllerAdvice*Tasks:*

1. Create a custom exception class named StudentNotFoundException.

Output:-





2400030211

2. Create a REST endpoint /student/{id} that retrieves student information and throws the custom exception when the given student ID is invalid.

Output:-

```
StudentContr...   StudentServi...   StudentNotFo...   ErrorRespons...
package com.klu.skillab9.exception;

import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;
    private String message;
    private int statusCode;

    public ErrorResponse(LocalDateTime timestamp, String message, int statusCode) {
        this.timestamp = timestamp;
        this.message = message;
        this.statusCode = statusCode;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    public String getMessage() {
        return message;
    }

    public int getStatusCode() {
        return statusCode;
    }
}
```

3. Create a GlobalExceptionHandler class using @ControllerAdvice to handle exceptions across the entire application.

Output:-

StudentContr... | StudentServi... | StudentNotFo... | GlobalExcep... | ErrorRespo

```java
package com.klu.skillab9.exception;

import java.time.LocalDateTime;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleStudentNotFound(StudentNotFoundException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.NOT_FOUND.value()
        );

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse> handleInvalidInput(InvalidInputException ex){

        ErrorResponse error = new ErrorResponse(
                LocalDateTime.now(),
                ex.getMessage(),
                HttpStatus.BAD_REQUEST.value()
        );

        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

2400035

4. Add a method with @ExceptionHandler(StudentNotFoundException.class) to return a meaningful error response along with an appropriate HTTP status code.

Output:-

```java
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

5. Add a second custom exception, such as InvalidInputException, and handle it inside the same @ControllerAdvice class.

Output:-

```java
package com.klu.skillab9.exception;

@SuppressWarnings("serial")
public class InvalidInputException extends RuntimeException {

    public InvalidInputException(String message) {
        super(message);
    }
}
```

6. Return a structured JSON response (fields like timestamp, message, statusCode) from the exception handler to make the error readable and consistent.

Output:-

GET http://localhost:8080/student/100

Body raw JSON

```
{
  "id": 1,
  "name": "Sathvik",
  "course": "CSE"
}
```

Body 404 Not Found

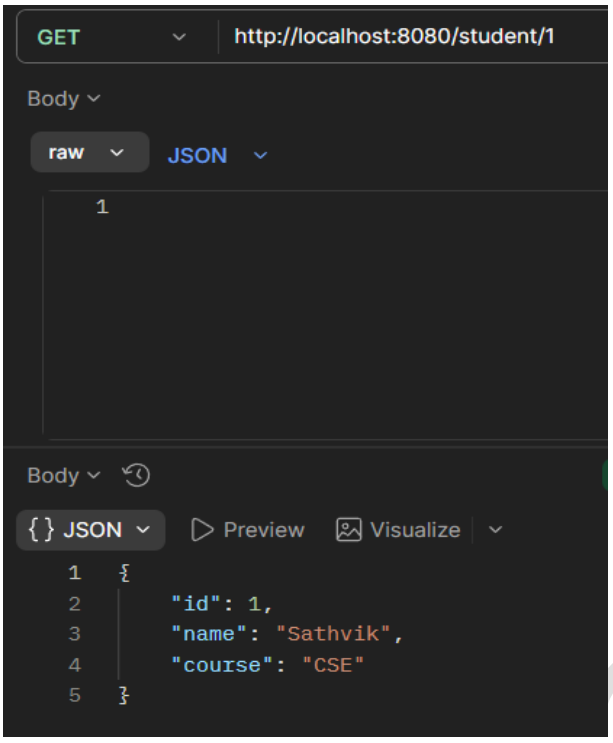
{} JSON Preview Debug with AI

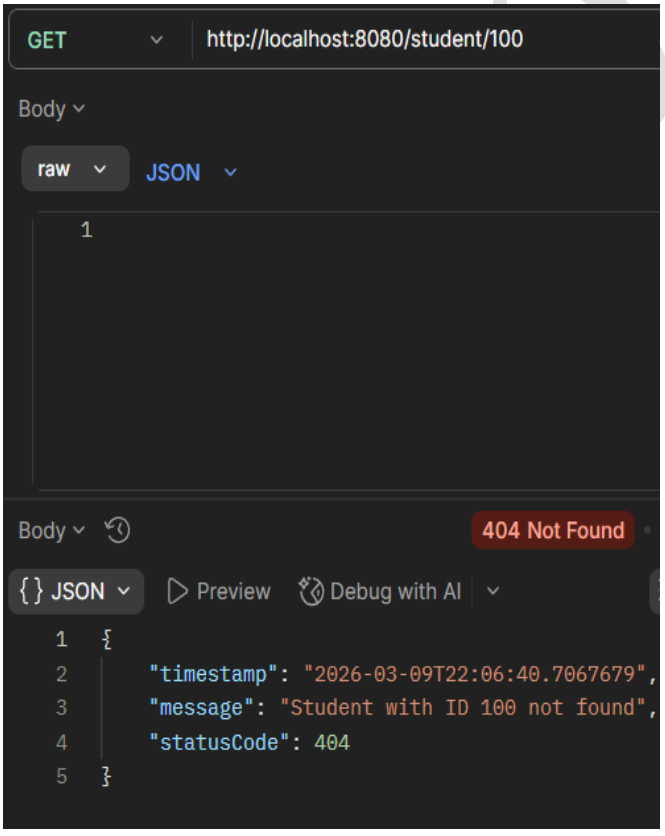
```
{
    "timestamp": "2026-03-09T22:03:33.4239534",
    "message": "Student with ID 100 not found",
    "statusCode": 404
}
```

030211

7. Test the API with valid and invalid student IDs and observe the difference in responses.

<u>Output:-</u>

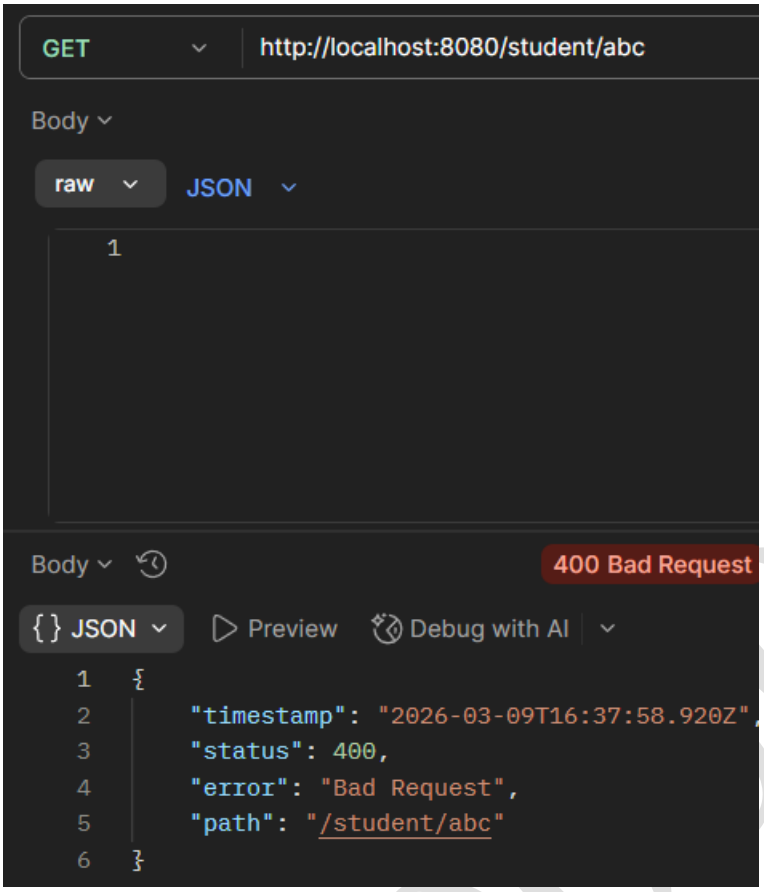




030211

8. Test an endpoint with invalid input format (e.g., sending text instead of a number) to trigger the second custom exception using Postman.

Output:-



9. Push the Spring Boot project to GitHub.

GITHUB REPO LINK :- https://github.com/hemalatharachakonda/2400030211-Skill_experiments.git

VIVA:

1. What is the purpose of @ControllerAdvice in Spring?

Answer:

@ControllerAdvice is used to handle exceptions globally across the whole Spring application. It allows developers to write centralized exception handling logic so that all controllers can share the same error handling methods.

2. How does @ExceptionHandler work inside a global handler?

Answer:

@ExceptionHandler is used to define methods that handle specific exceptions. When an exception occurs in any controller, Spring automatically calls the method annotated with @ExceptionHandler in the @ControllerAdvice class that matches that exception type.

3. Why do applications use global exception handling?

Answer:

2400030211

Applications use global exception handling to avoid repeating exception handling code in multiple controllers. It improves code readability, maintainability, and provides consistent error responses across the application.

4. What is the difference between custom exceptions and default exceptions?
Answer:

- **Default Exceptions:** Built-in exceptions provided by Java or Spring (e.g., NullPointerException, IOException).
- **Custom Exceptions:** User-defined exceptions created by developers to represent specific application errors.

5. Can a single @ControllerAdvice class handle multiple exceptions?
Answer:
Yes, a single @ControllerAdvice class can handle multiple exceptions. Multiple methods can be defined using @ExceptionHandler to handle different types of exceptions within the same class.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured:_____out of _____ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation:

2406211